Module 9: Solana Virtual Machine (SVM)

Runtime Architecture and Program Execution

What is the SVM?

Solana Virtual Machine

- **Sealevel Runtime**: Solana's parallel smart contract execution engine
- **BPF Programs**: Berkeley Packet Filter bytecode for programs
- **Parallel Processing**: Execute multiple transactions simultaneously
- **Resource Management**: Compute units and gas metering

Key Differences from EVM

- **Parallel Execution**: Multiple transactions can run at once
- **Account-Based**: Programs and data stored in accounts
- **No Gas Wars**: Priority fees instead of gas auctions

SVM Architecture

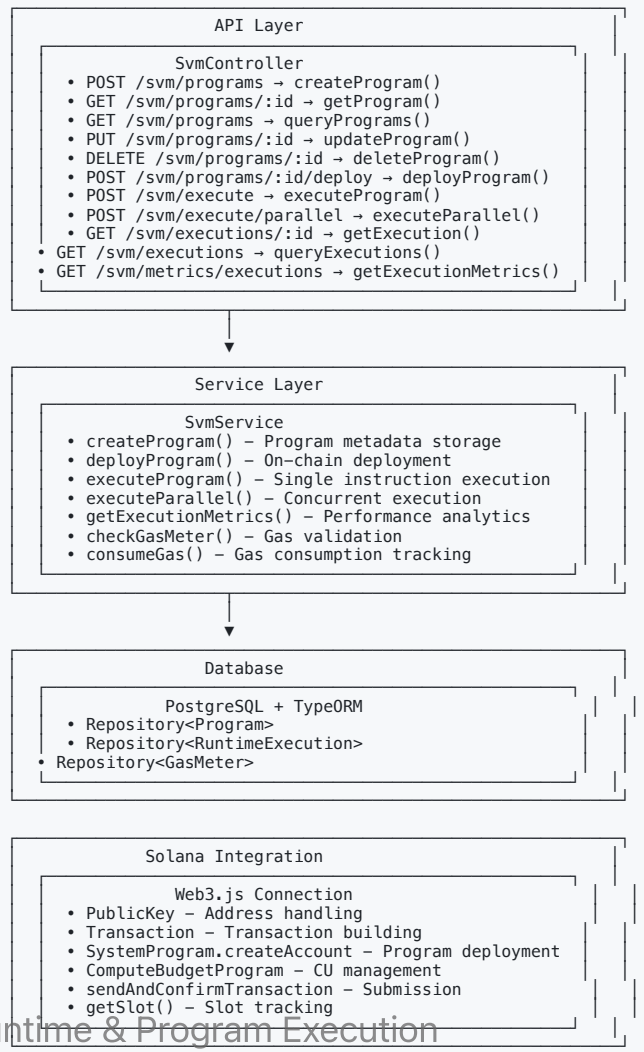
Core Components

- **Programs**: Executable bytecode stored in accounts
- **Runtime**: Sealevel execution environment
- **Compute Units**: Resource allocation and metering
- **Parallel Processing**: Concurrent transaction execution

Execution Model

- **Single-threaded Accounts**: No race conditions
- **Cross-program Calls**: Programs can invoke other programs
- **State Consistency**: Atomic transaction execution

Architecture Overview



Executing a Program

```
// Execute single instruction
const execution = await svmService.executeProgram({
  programId: program.programId,
  instruction: {
    accounts: [/* account metas */],
    data: instructionData,
    programId: program.programId
  },
  signers: [payerKeypair],
  computeUnitLimit: 100000
});

console.log('Execution signature:', execution.transactionId);
console.log('Compute units used:', execution.computeUnitsUsed);
```

Parallel Execution

```
// Execute multiple instructions in parallel
const results = await svmService.executeParallel({
  executions: [
    { programId: prog1, instruction: instr1 },
    { programId: prog2, instruction: instr2 },
    { programId: prog3, instruction: instr3 }
  ],
  continueOnFailure: true, // Continue if one fails
  maxConcurrent: 5 // Limit concurrency
});

// Check results
results.forEach((result, index) => {
  if (result.status === 'fulfilled') {
    console.log(`Execution ${index} succeeded:`, result.value.transactionId);
  } else {
    console.log(`Execution ${index} failed:`, result.reason);
  }
});
```

Compute Units (CU) Management

CU Concepts

- **Compute Units**: Measure of computational work
- **CU Price**: Priority fee per CU
- **CU Limit**: Maximum allowed per transaction
- **CU Budget**: Total available for transaction

Setting CU Limits

```
// Set compute unit limit and price
const modifyComputeUnits = ComputeBudgetProgram.setComputeUnitLimit({
  units: 100000
});

const addPriorityFee = ComputeBudgetProgram.setComputeUnitPrice({
  microLamports: 5000 // 0.000005 SOL per CU
});
```

Metrics and Monitoring

Execution Analytics

- **Performance Tracking**: Execution time measurement
- **Resource Usage**: CU consumption monitoring
- **Success Rates**: Execution outcome statistics
- **Cost Analysis**: Gas expenditure tracking
- **Trend Analysis**: Historical performance patterns

Monitoring Endpoints

- `GET /svm/metrics/executions` - Get execution metrics
- `GET /svm/programs/:programId/stats` - Get program statistics
- `GET /svm/runtime/info` - Get runtime information

Security and Validation

Execution Security

- **Program Verification**: Bytecode validation before execution
- **Access Control**: Owner permission checks
- **Gas Limits**: Resource bound enforcement
- **Account Validation**: Address verification
- **Error Isolation**: Failure containment

Validation Checks

- Program ownership verification
- Account existence validation
- Instruction data sanitization
- Resource limit enforcement

Performance Optimization

Optimization Strategies

- **Parallel Execution**: Run independent operations concurrently
- **CU Optimization**: Right-size compute unit limits
- **Batch Processing**: Combine related operations
- **Caching**: Cache program and account data
- **Connection Pooling**: Efficient RPC connections

Scalability Features

- **Horizontal Scaling**: Multiple SVM service instances
- **Load Balancing**: Distribute execution requests
- **Async Processing**: Non-blocking execution
- **Resource Monitoring**: Track and limit resource usage

Integration with Solana

Web3.js Integration

- **PublicKey**: Address handling and validation
- **Transaction**: Transaction building and serialization
- **SystemProgram**: Account creation and management
- **ComputeBudgetProgram**: CU limit and priority setting
- **sendAndConfirmTransaction**: Transaction submission and confirmation

Network Support

- **Local Validator**: Development and testing
- **Devnet/Testnet**: Staging environments
- **Mainnet**: Production deployment
- **RPC Rotation**: Failover and load balancing

Next Steps

Module 9 Implementation Tasks

- ✅ Program entity and CRUD operations
- ✅ Program deployment functionality
- ✅ Single program execution
- ✅ Parallel transaction execution
- ✅ Gas metering and tracking
- ✅ SVM runtime monitoring
- ✅ Local test validator setup
- ✅ Multi-network support

Related Modules

- **Module 10**: Cross-Program Invocations

Thank You!

Questions?

SVM enables parallel smart contract execution on Solana!

Back to Study Topics ⬅️